INSTITUT TEKNOLOGI DEL

Fakultas Informatika dan Teknik Elektro

Program Studi Sarjana Sistem Informasi • Sarjana Informatika

12S3101 — PPW

Ganjil 2026/2027

MODUL MINGGU 5

MODUL PRAKTIKUM MINGGU 05

IMPLEMENTASI POLA ARSITEKTUR MODEL-VIEW-CONTROLLER (MVC), REQUEST LIFECYCLE, ELOQUENT ORM, VALIDASI SERVER-SIDE, BLADE COMPONENT ENGINE, DAN ASSET BUNDLING VITE PADA LARAVEL 11.X

Mata Kuliah	: Pemrograman & Pengujian Web (12S3101)	**Dosen Pengampu**	: Chandro Pardede, S.Kom., M.Sc.
Keterkaitan Materi	: Transisi Arsitektur Decoupled ke MVC SSR	**Alokasi Waktu**	: 170' Lab + 360' Mandiri
Tahun Akademik	: Semester Ganjil 2026/2027	**Format Tugas**	: GitHub (ppw-2026-week5-[NIM])

I. Capaian Pembelajaran & Tujuan Praktikum

> **Sub-CPMK (SubCPMK12-02-09-04):** Mahasiswa mampu menerapkan pola arsitektur Model-View-Controller (MVC) dan alur kerja web framework modern dalam pengembangan aplikasi web berbasis server-side rendering secara terstruktur, modular, dan aman. [C3, A3, P3]

1.2 Indikator Ketercapaian Terukur

- **Pemodelan Arsitektur MVC Server-Side:** Mendekomposisi struktur aplikasi web ke dalam lapisan Model, View, dan Controller berlandaskan prinsip Separation of Concerns (SoC).
- **Analisis Siklus Hidup Request Laravel 11:** Menelusuri eksekusi HTTP Request dari titik masuk tunggal public/index.php, inisialisasi bootstrap/app.php, middleware pipeline, hingga HTTP Response.
- **Pengelolaan Lingkungan & Keamanan .env:** Mengonfigurasi berkas .env lokal dan mesin database (SQLite/MySQL) tanpa membocorkan kredensial rahasia ke repositori publik.
- **Desain Skema Data Berversi (Migrations):** Merancang skema tabel relasional berversi (Database Version Control) memanfaatkan schema builder tanpa menulis kueri SQL manual.
- **Otomasi Mock Data (Factories & Seeders):** Menyediakan data uji skala besar secara realistis memanfaatkan pustaka Faker untuk pengujian fitur aplikasi.
- **Implementasi Model Eloquent & Mitigasi N+1:** Menerapkan Active Record, proteksi mass assignment ($fillable), relasi One-to-Many, serta optimasi Eager Loading (with()) guna meniadakan N+1 Query Problem.

- **Routing RESTful & Resource Controller:** Mendaftarkan 7 aksi baku CRUD melalui Route::resource() dan menerapkan Implicit Route Model Binding.
- **Validasi Masukan Server-Side & Keamanan Web:** Menerapkan validasi ketat via $request->validate(), menangkal serangan CSRF via token @csrf, dan mencegah XSS otomatis.
- **Antarmuka Modular dengan Blade Components:** Membangun Master Shell Layout (<x-layout>), penanganan status flash session messages, pagination, dan direktif aman @forelse.
- **Tooling Artisan CLI & Asset Bundling Vite:** Mengoperasikan perintah scaffolding Artisan serta mengintegrasikan bundler Vite untuk kompilasi aset frontend modern.

II. Alat dan Bahan (Prerequisites)

Perangkat / Modul	Versi & Spesifikasi	Peran dalam Praktikum
PHP Runtime	PHP 8.2 atau 8.3 CLI	Ekstensi aktif: pdo_sqlite, pdo_mysql, mbstring, openssl, xml, curl, fileinfo.
Composer	Composer 2.6.x atau lebih baru	Manajer dependensi pustaka PHP dan registrasi PSR-4 Autoloading.
Node.js & NPM	Node.js 18.x / 20.x LTS & NPM	Eksekusi tooling frontend dan kompilasi aset antarmuka Vite.
Database Engine	SQLite / MySQL 8.0	Penyimpanan data relasional terstruktur (SQLite aktif secara bawaan).
Code Editor	Visual Studio Code	Ekstensi: PHP Intelephense, Laravel Blade Snippets, Laravel Artisan.
Version Control	Git CLI & Akun GitHub	Pelacakan riwayat commit kode dan pengumpulan tugas mandiri.

III. Landasan Teori dan Penjelasan Mendalam

3.1 Hakikat dan Filosofi Pola Arsitektur MVC

Pada era pengembangan web prosedural tradisional, instruksi basis data, kueri SQL, algoritma logika, dan sintaks HTML sering dicampur dalam satu berkas tunggal (dikenal sebagai Spaghetti Code). Pola ini menimbulkan kerapuhan tinggi di mana perubahan antarmuka berpotensi merusak logika basis data, menyulitkan unit testing otomatis, dan memicu konflik merge berkelanjutan dalam tim pengembang.

Pola arsitektur Model-View-Controller (MVC) menyelesaikan permasalahan tersebut dengan memisahkan aplikasi ke dalam tiga domain tanggung jawab yang terisolasi (Separation of Concerns):

- **Model (app/Models):** Merepresentasikan struktur data dan aturan bisnis, berkomunikasi langsung dengan database via ORM. Model bebas dari kode HTML.
- **View (resources/views):** Menyajikan data ke layar peramban melalui template engine Blade. View dilarang mengeksekusi kueri SQL langsung.
- **Controller (app/Http/Controllers):** Konduktor orkestrasi alur aplikasi: menerima HTTP Request, memanggil Model, dan mengirimkan data ke View yang tepat.

3.2 Anatomi Siklus Hidup HTTP Request Laravel 11

Pada Laravel 11, arsitektur internal disederhanakan secara signifikan. Seluruh request yang masuk dieksekusi melalui alur berurutan sebagai berikut:

- **1. public/index.php:** Titik masuk tunggal web server. Mencatat LARAVEL_START dan memuat Composer Autoloader.
- **2. bootstrap/app.php:** Inisialisasi Application Container IoC, pendaftaran rute deklaratif, dan konfigurasi pipa middleware.
- **3. Service Providers:** Mengeksekusi method register() untuk binding interface dan method boot() untuk inisialisasi layanan.
- **4. HTTP Middleware Pipeline:** Menyaring request secara berlapis (Onion Architecture), termasuk validasi token keamanan CSRF otomatis.
- **5. Routing & Controller Action:** Mencocokkan URL endpoint, mengikat model (Route Model Binding), dan mengeksekusi method controller.
- **6. Response Dispatch:** Mengembalikan payload HTML/JSON dengan status code yang valid, diikuti pembersihan memori sistem.

3.3 Model Layer: Eloquent ORM & Mitigasi N+1 Problem

Eloquent ORM mengimplementasikan pola Active Record di mana setiap tabel direpresentasikan oleh sebuah model PHP. Untuk mengamankan aplikasi dari kerentanan Mass Assignment, pengembang wajib mendefinisikan atribut yang boleh diisi melalui properti $fillable.

> **PERINGATAN OPTIMASI PERFORMA:** Bahaya Laten N+1 Query Problem: Ketika Anda meloop 100 data mahasiswa dan memanggil relasi program studinya secara Lazy Loading ($mhs->programStudi->nama), server akan mengeksekusi 101 kueri SQL! Solusinya adalah menerapkan Eager Loading: Mahasiswa::with('programStudi')->get() yang memangkas kueri menjadi tepat 2 kueri terpadu.

IV. Kegiatan Praktikum Terbimbing (Hands-on Labs)

LAB 1: Inisialisasi Proyek Laravel 11, Eksplorasi Lifecycle, & Database Environment (35 Menit)

Jalankan perintah berikut pada terminal untuk membuat kerangka kerja aplikasi SIM-Del:

Snippet Kode: Inisialisasi Proyek Laravel 11

```
# 1. Membuat proyek baru bernama sim-del
composer create-project laravel/laravel:^11.0 sim-del
cd sim-del


# 2. Menjalankan server pengembangan bawaan Artisan
php artisan serve
```

Periksa konfigurasi .env. Pastikan driver basis data menggunakan SQLite (default praktis Laravel 11):

Snippet Kode: Konfigurasi Berkas .env

```
APP_NAME="SIM-Del Mahasiswa"
APP_ENV=local
```

```
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=sqlite
# Database otomatis dibuat di database/database.sqlite
```

LAB 2: Pemodelan Skema Data & Relasi Eloquent (45 Menit)

Gunakan flag all-in-one Artisan untuk membuat Model, Migrasi, Factory, dan Resource Controller sekaligus:

Snippet Kode: Artisan Generator CLI

```
# Model ProgramStudi beserta file migrasinya
php artisan make:model ProgramStudi -m

# Model Mahasiswa beserta migrasi, factory, dan resource controller
php artisan make:model Mahasiswa -mfcr
```

Lengkapi skema migrasi tabel program_studis dan mahasiswas pada folder database/migrations/:

Snippet Kode: Skema Database Migrations

```
// database/migrations/xxxx_create_program_studis_table.php
Schema::create('program_studis', function (Blueprint $table) {
    $table->id();
    $table->string('kode', 10)->unique();
    $table->string('nama', 100);
    $table->string('jenjang', 10)->default('S1');
    $table->timestamps();
});

// database/migrations/xxxx_create_mahasiswas_table.php
Schema::create('mahasiswas', function (Blueprint $table) {
    $table->id();
    $table->string('nim', 10)->unique();
    $table->string('nama', 150);
    $table->string('email', 100)->unique();
    $table->foreignId('program_studi_id')->constrained('program_studis')->cascadeOnDelete();
    $table->decimal('ipk', 3, 2)->default(0.00);
    $table->timestamps();
});
```

Definisikan relasi kardinalitas One-to-Many pada app/Models/ProgramStudi.php dan app/Models/Mahasiswa.php:

Snippet Kode: Definisi Model & Relasi Eloquent

```
// app/Models/ProgramStudi.php
class ProgramStudi extends Model {
    use HasFactory;
    protected $fillable = ['kode', 'nama', 'jenjang'];
    public function mahasiswas(): HasMany {
        return $this->hasMany(Mahasiswa::class);
    }
}

// app/Models/Mahasiswa.php
```

```
class Mahasiswa extends Model {
    use HasFactory;
    protected $fillable = ['nim', 'nama', 'email', 'program_studi_id', 'ipk'];
    public function programStudi(): BelongsTo {
        return $this->belongsTo(ProgramStudi::class);
    }
}
```

Eksekusi migrasi skema dan suntikkan mock data seeder ke basis data:

Snippet Kode: Eksekusi Migrasi & Seeding

```
php artisan migrate:fresh --seed
```

LAB 3: Routing RESTful, Resource Controller, Validasi, & Blade UI (60 Menit)

Daftarkan 7 rute RESTful resource pada routes/web.php:

Snippet Kode: routes/web.php

```
use App\Http\Controllers\MahasiswaController;

Route::get('/', fn() => redirect()->route('mahasiswa.index'));
Route::resource('mahasiswa', MahasiswaController::class);
```

Lengkapi logika aksi CRUD dan validasi server-side pada app/Http/Controllers/MahasiswaController.php:

Snippet Kode: Aksi MahasiswaController.php

```
// 1. Tampilkan Semua Mahasiswa (Eager Loading + Pagination)
public function index() {
    $mahasiswas = Mahasiswa::with('programStudi')->latest()->paginate(10);
    return view('mahasiswa.index', compact('mahasiswas'));
}

// 2. Simpan Mahasiswa Baru ke Database (Validasi Ketat Sisi Server)
public function store(Request $request) {
    $validated = $request->validate([
        'nim'             => 'required|digits:8|unique:mahasiswas,nim',
        'nama'            => 'required|string|min:3|max:150',
        'email'           => 'required|email|unique:mahasiswas,email',
        'program_studi_id' => 'required|exists:program_studis,id',
        'ipk'             => 'required|numeric|between:0.00,4.00',
    ]);
    Mahasiswa::create($validated);
    return redirect()->route('mahasiswa.index')->with('success', 'Mahasiswa berhasil
ditambahkan!');
}
```

Buat Master Shell Layout resources/views/components/layout.blade.php bertema Del Royal Purple (#4C1D95):

Snippet Kode: resources/views/components/layout.blade.php

```
<!DOCTYPE html>
<html lang="id">
```

```
<head>
    <meta charset="UTF-8">
    <title>{{ $title ?? 'SIM-Del' }} — Institut Teknologi Del</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <style>
        .navbar-del { background-color: #4C1D95; }
        .btn-del { background-color: #4C1D95; color: #ffffff; }
        .btn-del:hover { background-color: #3B0764; color: #ffffff; }
    </style>
</head>
<body class="bg-light">
    <nav class="navbar navbar-expand-lg navbar-dark navbar-del shadow-sm mb-4">
        <div class="container">
            <a class="navbar-brand fw-bold" href="{{ route('mahasiswa.index') }}">SIM-Del
Mahasiswa</a>
        </div>
    </nav>
    <div class="container pb-5">
        @if(session('success'))
            <div class="alert alert-success alert-dismissible fade show">
                {{ session('success') }}
            </div>
        @endif
        {{ $slot }}
    </div>
</body>
</html>
```

LAB 4: Pengujian & Audit Kinerja Kueri SQL (30 Menit)

Tambahkan listener DB::listen pada app/Providers/AppServiceProvider.php untuk membuktikan Eager Loading mencegah N+1 problem:

Snippet Kode: Query Audit Listener

```
public function boot(): void {
    \Illuminate\Support\Facades\DB::listen(function ($query) {
        \Illuminate\Support\Facades\Log::info(sprintf(
            "[SQL AUDIT] (%.2f ms) %s", $query->time, $query->sql
        ));
    });
}
```

Segarkan peramban dan periksa storage/logs/laravel.log. Pastikan hanya ada 2 kueri SQL terpadu untuk mengambil seluruh daftar mahasiswa dan data program studinya.

V. Tugas Mandiri Mahasiswa (Individual Assignment)

> **JUDUL PENUGASAN MANDIRI:** Pembangunan Sistem Informasi Perpustakaan Kampus Del (SIPUS-Del): Implementasi Arsitektur MVC Lengkap, Relasi Eloquent Multi-Entitas, Validasi Form Server-Side, dan Audit Kueri Kinerja pada Laravel 11.x

Spesifikasi Teknis Wajib (Checklist Persyaratan)

- **Skema Data & Migrations (Bobot: 20%):** Tabel kategoris (kode_kategori, nama_kategori) dan tabel bukus (isbn, judul, penulis, penerbit, tahun_terbit, kategori_id foreign key, stok, sinopsis) dengan tipe data dan index yang presisi.
- **Model Eloquent & Relasi (Bobot: 20%):** Relasi Kategori hasMany Buku dan Buku belongsTo Kategori. Proteksi mass assignment dengan $fillable, serta implementasi Eager Loading (Buku::with('kategori')) di semua tampilan index.
- **Routing RESTful & Resource Controller (Bobot: 20%):** Mendaftarkan Route::resource untuk buku dan kategori. Mengimplementasikan 7 method CRUD lengkap dan menerapkan Implicit Route Model Binding.
- **Validasi Form Server-Side & CSRF (Bobot: 15%):** Validasi server ketat via $request->validate() (ISBN unik, judul minimal 5 karakter, stok minimal 0). Seluruh form wajib menyertakan @csrf dan method spoofing.
- **Blade Templating & Desain UI (Bobot: 15%):** Menggunakan master layout <x-layout> bertema Del Purple (#4C1D95), pagination aktif, notifikasi flash session, dan penanganan data kosong dengan @forelse.
- **Seeders & Bukti Audit SQL (Bobot: 10%):** Menyediakan Seeder dan Factory untuk minimal 5 kategori dan 20 buku realistis dengan Faker, serta melampirkan log kueri SQL pembuktian eager loading.

VI. Instruksi Git & Pengumpulan Tugas

1. Inisialisasi repositori Git dan gunakan konvensi pesan commit standar (Conventional Commits):

Snippet Kode: Konvensi Commit Git

```
git commit -m "feat(migration): create kategoris and bukus tables schema"
git commit -m "feat(model): define relationships and fillable attributes"
git commit -m "feat(controller): implement restful actions and server validation"
git commit -m "feat(views): construct blade component layout and crud templates"
git commit -m "perf(eager-loading): optimize sql queries to prevent n+1 problem"
```

2. DILARANG KERAS meng-commit berkas .env atau folder vendor/ ke GitHub. Pastikan terdaftar pada .gitignore dan sertakan berkas .env.example yang valid.

3. Format Nama Repositori Publik GitHub: ppw-2026-week5-[NIM]

📅 Batas Waktu Pengumpulan: Senin, 23 Februari 2026, Pukul 23.59 WIB.

VII. Rubrik Penilaian Praktikum Analitik

Kriteria Evaluasi	Bobot	Sangat Baik (85-100)	Baik (70-84)	Kurang (<70)
Arsitektur MVC & Routing RESTful	**20%**	Pemisahan M, V, C sangat tegas; 7 aksi RESTful terdaftar via resource; Route Model Binding presisi.	Struktur MVC terpisah baik; seluruh aksi RESTful berfungsi normal.	Logika kueri masih tertulis di View/Controller; rute tidak konsisten.
Model Relasional & Eloquent ORM	**25%**	Skema migrasi rapi; relasi hasMany &	Migrasi dan relasi Eloquent berfungsi	Skema migrasi tidak lengkap; kueri lambat

		belongsTo presisi; $fillable terproteksi; Eager Loading meniadakan N+1 problem.	baik; Eager Loading diterapkan di index.	karena masih terjadi N+1 problem.
Validasi Input & Keamanan Web	**20%**	Validasi server-side lengkap dan ketat; proteksi CSRF di seluruh form; bebas celah XSS & SQLi.	Validasi server-side berfungsi; pesan galat tampil; proteksi CSRF aktif.	Hanya mengandalkan validasi HTML5 sisi klien; tanpa proteksi token CSRF.
Blade Templating & Desain UI	**20%**	Master Shell <x-layout>; UI rapi bertema Del Purple; flash session interaktif; pagination & @forelse sempurna.	Layout Blade modular; UI rapi dan responsif; notifikasi flash berfungsi.	Duplikasi template di tiap halaman; tampilan berantakan; tanpa komponen.
Seeders, Logging, Git & Laporan	**15%**	Mock data Faker realistis; log audit SQL pembuktian Eager Loading lengkap; commit Git teratur berstandar; README detail.	Seeder berfungsi; dokumentasi README lengkap beserta panduan instalasi.	Seeder gagal; .env bocor ke repositori publik; README kosong/tidak jelas.